

Module 3 - C#ADO.NET

Folder Creation in Terminal

```
dotnet new console -n CSharpExercises  
cd CSharpExercises
```

```
dotnet new console -n Exercise02  
dotnet new console -n Exercise04  
dotnet new console -n Exercise05  
dotnet new console -n Exercise06  
dotnet new console -n Exercise07  
dotnet new console -n Exercise10  
dotnet new console -n Exercise13  
dotnet new console -n Exercise19  
dotnet new console -n Exercise21  
dotnet new console -n Exercise22  
dotnet new console -n Exercise23
```

I have to update my code in program.cs and run it using:

```
cd Exercise02  
dotnet run  
to go to next program or folder:  
cd ..  
cd Exercise04  
dotnet run
```

or compile manually:

```
csc Exercise02.cs  
Exercise02.exe
```

C Sharp Exercises

2. Write a C# program to Explore Value vs Reference Types

Objective: Understand the differences between value types and reference types.

Instructions:

- Define variables using both value types (e.g., int, double) and reference types (e.g., string, a custom class).
- Create methods that attempt to modify these variables.
- Print the values before and after method calls to illustrate the behavior of value types and reference types.

Source Code:

```
class Person  
{  
    public string Name;  
}
```

```

class Program
{
    static void ChangeValue(int x)
    {
        x = 100;
    }

    static void ChangeReference(Person p)
    {
        p.Name = "John";
    }

    static void Main()
    {
        int num = 10;
        Person person = new Person { Name = "Alice" };

        Console.WriteLine($"Before: {num}");
        ChangeValue(num);
        Console.WriteLine($"After: {num}");

        Console.WriteLine($"Before: {person.Name}");
        ChangeReference(person);
        Console.WriteLine($"After: {person.Name}");
    }
}

```

```

TERMINAL ... powershell - Exercise04 + v [] [X] ...
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpExercises> cd Exercise02
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpExercises\Exercise02> dotnet run
C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpExercises\Exercise02\Program.cs(3,19): warning CS8618: Non-nullable field 'Name' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the field as nullable.
Before: 10
After: 10
Before: Alice
After: John

```

4. Write a C# program to Demonstrate Type Inference with var and new()

Objective: Understand the use of type inference in C#.

Instructions:

- Create variables using var and new() for different types (e.g., int, string, a custom class).
- Print the types and values of each variable.
- Discuss when type inference is beneficial and when it might affect readability.

Source Code:

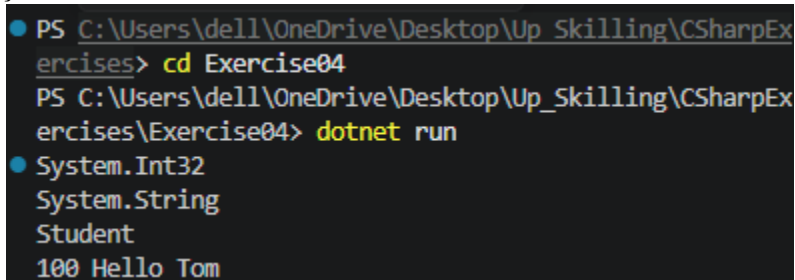
```
class Student
{
    public string Name { get; set; } = "";
}

class Program
{
    static void Main()
    {
        var number = 100;
        var text = "Hello";

        Student s = new()
        {
            Name = "Tom"
        };

        Console.WriteLine(number.GetType());
        Console.WriteLine(text.GetType());
        Console.WriteLine(s.GetType());

        Console.WriteLine($"{number} {text} {s.Name}");
    }
}
```



```
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpExercises> cd Exercise04
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpExercises\Exercise04> dotnet run
System.Int32
System.String
Student
100 Hello Tom
```

5. Write a C# program to Perform Conditional Logic for Grade Calculation

Objective: Use conditional statements to calculate and display grades.

Instructions:

- Accept a score from the user and use if, else if, else, and switch to determine the grade.
- Use pattern matching in the switch statement to handle different score ranges more concisely.

Source Code:

```
class Program
{
    static void Main()
    {
        Console.Write("Enter Score: ");
        int score = Convert.ToInt32(Console.ReadLine());
    }
}
```

```

string grade = score switch
{
    >= 90 => "A",
    >= 80 => "B",
    >= 70 => "C",
    >= 60 => "D",
    _ => "F"
};

Console.WriteLine($"Grade: {grade}");
}
}

```

```

ercises\Exercise06> cd ..
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpEx>
ercises> cd Exercise05
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpEx>
ercises\Exercise05> dotnet run
Enter Score: 10
Grade: F
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpEx>
ercises\Exercise05> dotnet run
Enter Score: 57
Grade: F
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpEx>
ercises\Exercise05> dotnet run
Enter Score: 95
Grade: A
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpEx>
ercises\Exercise05>

```

6. Write a C# program to Loop Through an Array with Different Loop Types

Objective: Practice iterating over arrays using various loops.

Instructions:

- Declare an array of integers with at least 5 elements.
- Use a for, foreach, while, and do-while loop to iterate through the array.
- Include conditions within the loops to skip or stop based on specific values.

Source Code:

```

class Program
{
    static void Main()
    {
        int[] nums = { 10, 20, 30, 40, 50 };

        Console.WriteLine("For Loop");
        for (int i = 0; i < nums.Length; i++)
            Console.WriteLine(nums[i]);
    }
}

```

```
Console.WriteLine("Foreach Loop");  
foreach (int n in nums)  
    Console.WriteLine(n);
```

```
Console.WriteLine("While Loop");  
int j = 0;  
while (j < nums.Length)  
{  
    Console.WriteLine(nums[j]);  
    j++;  
}
```

```
Console.WriteLine("Do While Loop");  
int k = 0;  
do  
{  
    Console.WriteLine(nums[k]);  
    k++;  
} while (k < nums.Length);  
}
```

```

ercises> cd Exercise06
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpEx
ercises\Exercise06> dotnet run
For Loop
10
20
30
40
50
Foreach Loop
10
20
30
40
50
While Loop
10
20
30
40
50
Do While Loop
10
20
30
40
50
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpEx
ercises\Exercise06>

```

7. Write a C# program to Implement Method Overloading

Objective: Demonstrate method overloading.

Instructions:

- Define a method named CalculateTotal that accepts different types/number of parameters (e.g., two integers, three doubles).
- Call each version of the method from Main() and display the results to demonstrate method overloading.

Source Code:

```

class Program
{
    static int CalculateTotal(int a, int b)
    {
        return a + b;
    }

    static double CalculateTotal(double a, double b, double c)
    {
        return a + b + c;
    }
}

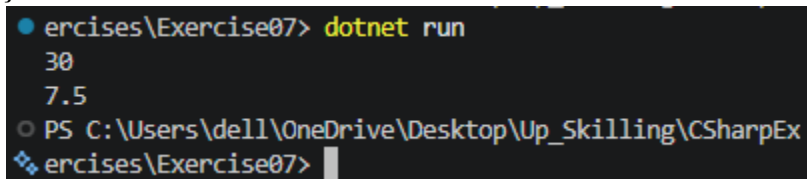
```

```

    }

    static void Main()
    {
        Console.WriteLine(CalculateTotal(10, 20));
        Console.WriteLine(CalculateTotal(1.5, 2.5, 3.5));
    }
}

```



```

ercises\Exercise07> dotnet run
30
7.5
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpEx
ercises\Exercise07>

```

10. Write a C# program to Demonstrate OOP Basics with Constructors

Objective: Implement object-oriented concepts using constructors.

Instructions:

- Create a Car class with properties for Make, Model, and Year.
- Implement both a default constructor and a parameterized constructor.
- Create two objects of the class using both constructors and display their values.

Source Code:

```

class Car
{
    public string Make;
    public string Model;
    public int Year;

    public Car()
    {
        Make = "Unknown";
        Model = "Unknown";
        Year = 0;
    }

    public Car(string make, string model, int year)
    {
        Make = make;
        Model = model;
        Year = year;
    }
}

class Program
{
    static void Main()
    {
        Car c1 = new Car();
    }
}

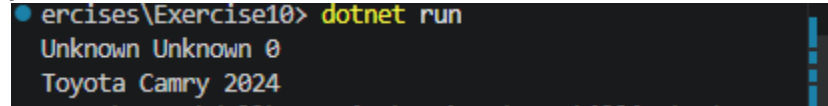
```

```

        Car c2 = new Car("Toyota", "Camry", 2024);

        Console.WriteLine($"{c1.Make} {c1.Model} {c1.Year}");
        Console.WriteLine($"{c2.Make} {c2.Model} {c2.Year}");
    }
}

```



```

ercises\Exercise10> dotnet run
Unknown Unknown 0
Toyota Camry 2024

```

13. Write a C# program to Create and Use Records with init Properties

Objective: Explore the use of records and init properties in C# 9 and 12.

Instructions:

- Define an immutable Employee record with init properties.
- Instantiate the record and attempt to modify its properties using the with expression to create a modified copy.
- Print the properties to verify that the original record is unchanged.

Source Code:

```

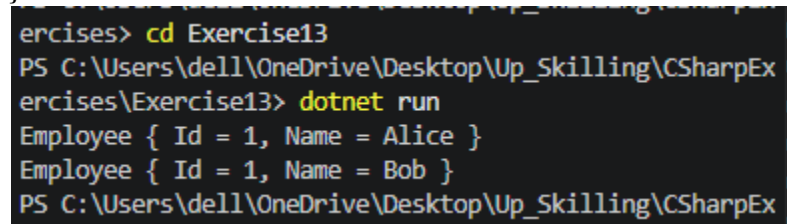
public record Employee
{
    public int Id { get; init; }
    public string Name { get; init; } = "";
}

class Program
{
    static void Main()
    {
        Employee e1 = new()
        {
            Id = 1,
            Name = "Alice"
        };

        Employee e2 = e1 with { Name = "Bob" };

        Console.WriteLine(e1);
        Console.WriteLine(e2);
    }
}

```



```

ercises> cd Exercise13
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpEx
ercises\Exercise13> dotnet run
Employee { Id = 1, Name = Alice }
Employee { Id = 1, Name = Bob }
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpEx

```

19. Write a C# program to Work with Lists and Dictionaries

Objective: Understand how to work with List<T> and Dictionary<K, V> collections.

Instructions:

- Create a List<string> and a Dictionary<int, string> and populate them with values.
- Use foreach loops to iterate over and print the entries.
- Perform basic operations like adding and removing items.

Source Code:

```
using System.Collections.Generic;
```

```
class Program
{
    static void Main()
    {
        List<string> fruits = new()
        {
            "Apple",
            "Banana",
            "Mango"
        };

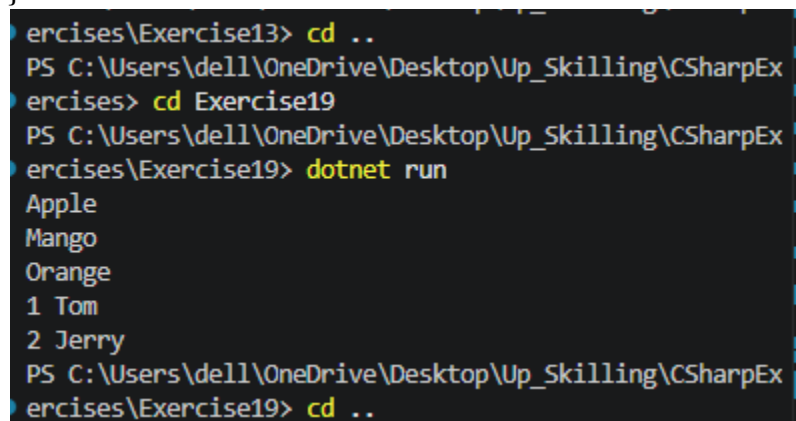
        fruits.Add("Orange");
        fruits.Remove("Banana");

        foreach (var item in fruits)
            Console.WriteLine(item);

        Dictionary<int, string> students = new();

        students.Add(1, "Tom");
        students.Add(2, "Jerry");

        foreach (var s in students)
            Console.WriteLine($"{s.Key} {s.Value}");
    }
}
```



```
ercises\Exercise13> cd ..
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpExercises> cd Exercise19
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpExercises\Exercise19> dotnet run
Apple
Mango
Orange
1 Tom
2 Jerry
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpExercises\Exercise19> cd ..
```

21. Write a C# program to Use Pattern Matching with is and switch

Objective: Explore the power of pattern matching with is and switch.

Instructions:

- Create a method that accepts an object as a parameter.
- Use pattern matching with is to check the type of the object and display appropriate information.
- Use an enhanced switch statement to perform operations based on object types.

Source Code:

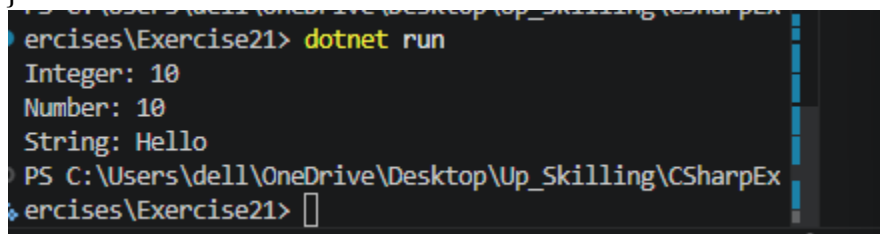
```
class Program
{
    static void Check(object obj)
    {
        if (obj is int number)
            Console.WriteLine($"Integer: {number}");

        switch (obj)
        {
            case string s:
                Console.WriteLine($"String: {s}");
                break;

            case int n:
                Console.WriteLine($"Number: {n}");
                break;

            default:
                Console.WriteLine("Unknown");
                break;
        }
    }

    static void Main()
    {
        Check(10);
        Check("Hello");
    }
}
```



```
ercises\Exercise21> dotnet run
Integer: 10
Number: 10
String: Hello
PS C:\Users\de11\OneDrive\Desktop\Up_Skilling\CSharpEx
ercises\Exercise21>
```

22. Write a C# program to Create and Deconstruct Tuples

Objective: Learn how to use tuples for returning multiple values.

Instructions:

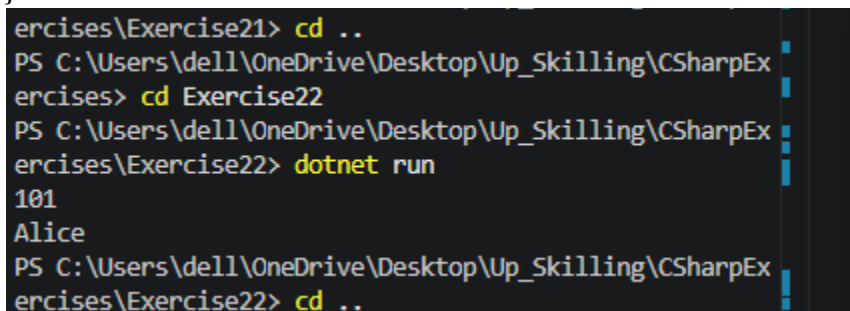
- Create a method that returns a tuple with two values: an int and a string.
- In Main(), deconstruct the tuple and print the individual values.

Source Code:

```
class Program
{
    static (int, string) GetData()
    {
        return (101, "Alice");
    }

    static void Main()
    {
        var (id, name) = GetData();

        Console.WriteLine(id);
        Console.WriteLine(name);
    }
}
```



```
ercises\Exercise21> cd ..
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpEx
ercises> cd Exercise22
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpEx
ercises\Exercise22> dotnet run
101
Alice
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpEx
ercises\Exercise22> cd ..
```

23. Write a C# program to Simulate Async File Upload with Exception Handling

Objective: Understand how to work with asynchronous methods and exception handling.

Instructions:

- Create an asynchronous method that simulates a file upload operation by waiting for 3 seconds.
- Return a success message after the delay, and use try-catch to handle any exceptions during the upload process.

Source Code:

```
class Program
{
    static async Task<string> UploadFileAsync()
    {
        await Task.Delay(3000);
        return "File Uploaded Successfully";
    }

    static async Task Main()
    {

```

```
try
{
    string result = await UploadFileAsync();
    Console.WriteLine(result);
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
}
```

```
ercises> cd Exercise23
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpEx
ercises\Exercise23> dotnet run
File Uploaded Successfully
PS C:\Users\dell\OneDrive\Desktop\Up_Skilling\CSharpEx
ercises\Exercise23> 
```